# Inverse Cosine Similarity: Algorithm B

Construction, Theory, and Applications

Soumadeep Ghosh

Kolkata, India

## Contents

**1 Introduction** **2**

**2 Background and Notation** **3**
2.1 Cosine Similarity . . . 3
2.2 Orthonormal Basis and QR Decomposition . . . 3
2.3 Householder Reflections . . . 3

**3 The Inverse Cosine Similarity Algorithm B: Precomputed Basis Method** **3**
3.1 Setup Phase: Precomputing the ONB . . . 3
3.2 Query Phase: $\mathcal{O}(n)$ Output with $\mathcal{O}(1)$ Arithmetic . . . 4
3.3 Batch Query Phase . . . 4

**4 Theoretical Analysis** **4**
4.1 Correctness . . . 4
4.2 Correctness of the Householder Construction . . . 5
4.3 Complexity . . . 5
4.4 Amortised Query Cost . . . 5
4.5 Space–Time Trade-off . . . 5

**5 Geometric Interpretation** **6**

**6 Comparison with Algorithm A** **6**
6.1 Operation Counts . . . 6
6.2 Memory Footprint . . . 6
6.3 Break-Even Analysis . . . 6

**7 Numerical Results** **7**
7.1 Accuracy . . . 7
7.2 Runtime Comparison . . . 7

**8 Applications** **8**
8.1 Finance: High-Frequency Correlation Streaming . . . 8
8.2 AI/ML: Embedding Space Exploration . . . 8
8.3 Statistics: Monte Carlo Angular Sampling . . . 8

**9 Extensions** **8**
9.1 Multiple Anchor Vectors . . . 8
9.2 Streaming Updates via Rank-One Modifications . . . 8
9.3 GPU Parallelisation . . . 9

**10 Conclusion** **9**

**A Full Householder Expansion** **9**

# List of Figures


1 (a) In $\mathbb{R}^2$, Algorithm B blends $q_1 = \hat{\mathbf{u}}$ with the precomputed $q_2$ to place $\mathbf{v}$ at angle $\theta = \arccos(s)$ from $\mathbf{u}$. (b) In $\mathbb{R}^3$, distinct basis columns $q_2, q_3$ (and their linear combinations) span the entire angular shell $\mathcal{V}(s)$ — a circle of radius $\sqrt{1-s^2}$ in the plane $\{x : q_1^\top x = s\}$. 6

2 Total runtime (setup + queries) for Algorithm A and Algorithm B as a function of query count $M$ for $n = 512$. Algorithm B becomes faster after the break-even point $M^* \approx 771$. . 7

3 Per-query runtime of Algorithm A (circles) and Algorithm B query phase (squares) across dimensions. Algorithm B achieves a consistent $\approx 1.7\times$ speedup owing to the elimination of dot products and random number generation. . . . . 8


# List of Algorithms


1 Algorithm B — Setup Phase . . . . . 3

2 Algorithm B — Query Phase . . . . . 4

3 Algorithm B — Batch Query Phase . . . . . 4


# List of Tables


1 Per-query floating-point operation counts ($n$-dimensional vectors). "RNG" = random number generator calls. . . . . 6

2 Memory footprint in units of $n$ floating-point numbers (each 8 bytes for double precision). 6

3 Achieved cosine similarity $\hat{s}$ and error $|s - \hat{s}|$ for Algorithm B (explicit $Q$), averaged over $10^4$ independent queries. . . . . 7


# 1 Introduction

The original Algorithm A [1] solves the inverse cosine similarity problem in $\mathcal{O}(n)$ time per query by performing a Gram–Schmidt orthogonalisation at each invocation. In many practical settings — such as real-time embedding perturbation in recommendation engines [4], high-frequency portfolio stress-testing [6], or adversarial example generation in neural networks [3] — the anchor vector $\mathbf{u}$ is *fixed* while the target similarity $s$ varies across thousands or millions of queries. In this regime the per-query Gram–Schmidt step is wasteful.

This paper presents **Algorithm B**, which trades storage for speed by precomputing and caching a full orthonormal basis for $\mathbb{R}^n$ during a one-time $\mathcal{O}(n^2)$ setup phase. Subsequent queries for *any* target $s \in [-1, 1]$ are resolved by a simple two-term linear combination requiring only $\mathcal{O}(n)$ *work* on cached data — with no floating-point divisions, norm computations, or random draws at query time. When the same basis column is reused across queries, the effective amortised cost per query is $\mathcal{O}(1)$ scalar operations after an $\mathcal{O}(n)$ final assembly.

The key contributions are:

1. A precomputed $n \times n$ orthonormal matrix $Q$ whose first column is $\hat{\mathbf{u}}$ (Section 3.1).
2. An $\mathcal{O}(1)$-amortised query procedure that selects a basis column and forms the output by a two-scalar blend (Section 3.2).
3. A batch extension that generates $K$ distinct outputs for $K$ target similarities in $\mathcal{O}(Kn)$ time after the setup phase (Section 3.3).
4. Formal proofs of correctness, termination, and complexity (Section 4).
5. An explicit space–time trade-off characterisation (Section 4.5).

The paper ends with "The End"

# 2 Background and Notation

## 2.1 Cosine Similarity

**Definition 2.1** (Cosine Similarity)**.** *For* $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$*,*

$$\mathrm{cs}(\mathbf{u}, \mathbf{v}) \;=\; \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \;\in\; [-1, 1]. \tag{1}$$

## 2.2 Orthonormal Basis and QR Decomposition

**Definition 2.2** (Orthonormal Basis)**.** *A set* $\{q_1, \ldots, q_n\} \subset \mathbb{R}^n$ *is an* orthonormal basis *(ONB) if* $q_i^\top q_j = \delta_{ij}$ *for all* $i, j$*, where* $\delta_{ij}$ *is the Kronecker delta. Equivalently, the matrix* $Q = [q_1 \mid \cdots \mid q_n] \in \mathbb{R}^{n \times n}$ *satisfies* $Q^\top Q = QQ^\top = I_n$*.*

**Definition 2.3** (Reduced QR Decomposition)**.** *For* $\mathbf{u} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$*, the* reduced QR decomposition *of* $\mathbf{u}$ *is the factorisation* $\mathbf{u} = QR$ *where* $Q \in \mathbb{R}^{n \times n}$ *is orthogonal and* $R \in \mathbb{R}^{n \times n}$ *is upper triangular. For our purposes we only need the first column:* $q_1 = \hat{\mathbf{u}} = \mathbf{u}/\|\mathbf{u}\|$*.*

## 2.3 Householder Reflections

The most numerically stable method for constructing a full ONB with a prescribed first column is via *Householder reflections* [2]. Given $\hat{\mathbf{u}} \in \mathbb{R}^n$, define

$$\mathbf{a} = e_1 \in \mathbb{R}^n, \qquad \mathbf{x} = \hat{\mathbf{u}} - e_1, \qquad H = I - \frac{2\mathbf{x}\mathbf{x}^\top}{\|\mathbf{x}\|^2}, \tag{2}$$

so that $He_1 = \hat{\mathbf{u}}$. The columns of $H^\top = H$ (since $H$ is symmetric and orthogonal) then form an ONB with $q_1 = \hat{\mathbf{u}}$. The full matrix $Q$ of Algorithm B is constructed as $Q = H$ (or a product of Householder reflectors when $\hat{\mathbf{u}} = e_1$ exactly).

# 3 The Inverse Cosine Similarity Algorithm B: Precomputed Basis Method

Algorithm B has two distinct phases: a **setup phase** executed once per anchor vector, and a **query phase** executed once per target similarity.

## 3.1 Setup Phase: Precomputing the ONB

**Algorithm 1** Algorithm B — Setup Phase

**Require:** $\mathbf{u} \in \mathbb{R}^n$, $\mathbf{u} \neq \mathbf{0}$, $n \geq 2$
**Ensure:** Orthonormal matrix $Q \in \mathbb{R}^{n \times n}$ with $q_1 = \hat{\mathbf{u}}$

```
û ← u/‖u‖                                   ▷ O(n)
if ‖û − e_1‖ < ε then
    Q ← [e_2 | e_1 | e_3 | ··· | e_n]       ▷ Swap to avoid degenerate Householder
else
    x ← û − e_1                             ▷ Householder vector, O(n)
    τ ← 2/(x^⊤ x)                           ▷ Scalar, O(n)
    Store (x, τ) implicitly or expand:
    Q ← I_n − τ x x^⊤                       ▷ O(n^2) time, O(n^2) space
end if
return Q
```

**Remark 3.1.** *The matrix* $Q$ *may be stored* implicitly *as the pair* $(\mathbf{x}, \tau) \in \mathbb{R}^n \times \mathbb{R}$*, using only* $\mathcal{O}(n)$ *space. Applying* $Q$ *to any vector then costs* $\mathcal{O}(n)$*. Storing* $Q$ explicitly *costs* $\mathcal{O}(n^2)$ *space but reduces query-time column access to* $\mathcal{O}(1)$ *— this is the storage–speed trade-off explored in Section 4.5.*

### 3.2 Query Phase: $\mathcal{O}(n)$ Output with $\mathcal{O}(1)$ Arithmetic

**Algorithm 2** Algorithm B — Query Phase

**Require:** Precomputed $Q = [q_1 \mid \cdots \mid q_n]$ with $q_1 = \hat{\mathbf{u}}$; target $s \in [-1, 1]$; column index $j \in \{2, \ldots, n\}$ (default $j = 2$)
**Ensure:** $\mathbf{v} \in \mathbb{R}^n$ with $\mathrm{cs}(\mathbf{u}, \mathbf{v}) = s$, $\|\mathbf{v}\| = 1$

```
α ← s                                  ▷ Parallel weight, O(1)
β ← √(1 − s²)                          ▷ Perpendicular weight, O(1)
v ← α q_1 + β q_j                      ▷ Two-column blend, O(n)
return v
```

The only floating-point operations at query time are:

- One square root: $\beta = \sqrt{1 - s^2}$.
- $2n$ multiplications and $n$ additions for the linear combination.

There are **no** dot products, no divisions, and no random number generation. Compared to Algorithm A, the query-time operation count drops from $\approx 5n$ (Algorithm A: norm, dot, subtract, norm, linear combination) to $\approx 3n$ (Algorithm B: two scalar-vector multiplies plus vector addition), with all memory accesses hitting pre-warmed cache lines.

### 3.3 Batch Query Phase

**Algorithm 3** Algorithm B — Batch Query Phase

**Require:** Precomputed $Q$; targets $s_1, \ldots, s_K \in [-1, 1]$
**Ensure:** $\mathbf{v}_k$ with $\mathrm{cs}(\mathbf{u}, \mathbf{v}_k) = s_k$ for $k = 1, \ldots, K$

```
for k = 1 to K do
    j_k ← ((k mod (n − 1)) + 2)        ▷ Cycle through basis columns
    v_k ← s_k q_1 + √(1 − s_k²) q_{j_k}   ▷ O(n) per query
end for
return v_1, ..., v_K
```

By cycling through distinct columns $q_2, \ldots, q_n$, consecutive outputs are guaranteed to be mutually non-parallel in their perpendicular components, maximising angular spread on the solution manifold $\mathcal{V}(s)$.

## 4 Theoretical Analysis

### 4.1 Correctness

**Theorem 4.1** (Correctness of Algorithm B)**.** *Let $Q$ be the output of Algorithm B Setup (Algorithm 1) and $\mathbf{v}$ the output of Algorithm B Query (Algorithm 2). Then $\mathrm{cs}(\mathbf{u}, \mathbf{v}) = s$ and $\|\mathbf{v}\| = 1$.*

*Proof.* By construction, $Q$ is orthogonal with $q_1 = \hat{\mathbf{u}}$, so $q_1^\top q_j = \delta_{1j} = 0$ for $j \geq 2$ and $\|q_j\| = 1$ for all $j$. For $\mathbf{v} = \alpha q_1 + \beta q_j$ with $j \geq 2$:

$$\mathbf{u}^\top \mathbf{v} = \|\mathbf{u}\| \, \hat{\mathbf{u}}^\top (\alpha q_1 + \beta q_j) = \|\mathbf{u}\| \left( \alpha \underbrace{q_1^\top q_1}_{1} + \beta \underbrace{q_1^\top q_j}_{0} \right) = \alpha \, \|\mathbf{u}\|$$

Also $\|\mathbf{v}\|^2 = \alpha^2 \|q_1\|^2 + 2\alpha\beta q_1^\top q_j + \beta^2 \|q_j\|^2 = \alpha^2 + \beta^2 = s^2 + (1 - s^2) = 1$. Therefore

$$\mathrm{cs}(\mathbf{u}, \mathbf{v}) = \frac{\alpha \, \|\mathbf{u}\|}{\|\mathbf{u}\| \cdot 1} = \alpha = s. \quad \square$$

## 4.2 Correctness of the Householder Construction

**Lemma 4.2** (Householder Preserves $\hat{\mathbf{u}}$)**.** *The matrix $H = I - \tau\mathbf{x}\mathbf{x}^\top$ with $\mathbf{x} = \hat{\mathbf{u}} - e_1$ and $\tau = 2/\|\mathbf{x}\|^2$ satisfies $He_1 = \hat{\mathbf{u}}$ and $H^\top H = I_n$.*

*Proof.* **(Orthogonality.)** $H^\top H = (I - \tau\mathbf{x}\mathbf{x}^\top)^2 = I - 2\tau\mathbf{x}\mathbf{x}^\top + \tau^2(\mathbf{x}^\top\mathbf{x})\mathbf{x}\mathbf{x}^\top = I - 2\tau\mathbf{x}\mathbf{x}^\top + \tau^2 \cdot \frac{2}{\tau}\mathbf{x}\mathbf{x}^\top = I$. **($He_1 = \hat{\mathbf{u}}$.)** $He_1 = e_1 - \tau(\mathbf{x}^\top e_1)\mathbf{x} = e_1 - \tau(\hat{u}_1 - 1)\mathbf{x}$. Now $\|\mathbf{x}\|^2 = \|\hat{\mathbf{u}}\|^2 - 2\hat{u}_1 + 1 = 2 - 2\hat{u}_1$, so $\tau = 1/(1 - \hat{u}_1)$. Thus $He_1 = e_1 - \frac{\hat{u}_1 - 1}{1 - \hat{u}_1}(\hat{\mathbf{u}} - e_1) = e_1 + (\hat{\mathbf{u}} - e_1) = \hat{\mathbf{u}}$. □

## 4.3 Complexity

**Theorem 4.3** (Complexity of Algorithm B)**.** *Algorithm B has the following complexity profile.*

| *Phase* | *Time* | *Additional Space* | *Divisions / RNG calls* |
|---|---|---|---|
| *Setup (implicit Q)* | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | *1 / 0* |
| *Setup (explicit Q)* | $\mathcal{O}(n^2)$ | $\mathcal{O}(n^2)$ | *1 / 0* |
| *Query (explicit Q)* | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ | *0 / 0* |
| *Query (implicit Q)* | $\mathcal{O}(n)$ | $\mathcal{O}(1)$ | *0 / 0* |
| *Batch K (explicit)* | $\mathcal{O}(Kn)$ | $\mathcal{O}(n)$ | *0 / 0* |
| *Algorithm A (per query)* | $\mathcal{O}(n)$ | $\mathcal{O}(n)$ | *2 / 1* |

*Proof.* **Setup.** Computing $\hat{\mathbf{u}}$ costs $\mathcal{O}(n)$ (one dot product, one division, one scalar-vector multiply). Computing $\mathbf{x} = \hat{\mathbf{u}} - e_1$ and $\tau$ costs $\mathcal{O}(n)$. Expanding $Q = I - \tau\mathbf{x}\mathbf{x}^\top$ costs $\mathcal{O}(n^2)$ time and space (outer product). **Query.** Column access $q_j$ from an explicit $Q$ is a pointer offset, $\mathcal{O}(1)$. Two scalar-vector multiplies and one addition cost $\mathcal{O}(n)$. One square root is $\mathcal{O}(1)$. No dot products, no norm computations, no RNG. □

## 4.4 Amortised Query Cost

**Corollary 4.4** (Amortised Cost)**.** *For $M$ queries against the same anchor $\mathbf{u}$, the amortised cost per query of Algorithm B (explicit) is*

$$T_{amortised}(n, M) = \frac{T_{setup} + M \cdot T_{query}}{M} = \frac{\mathcal{O}(n^2)}{M} + \mathcal{O}(n) \xrightarrow{M \to \infty} \mathcal{O}(n). \tag{3}$$

*The setup cost is fully amortised after $M = \Theta(n)$ queries.*

*Proof.* Direct substitution of $T_{\text{setup}} = cn^2$ and $T_{\text{query}} = dn$ for constants $c, d > 0$ into (3): $T_{\text{amortised}} = cn^2/M + dn$. For $M \geq cn/d$ the first term is $\mathcal{O}(n)$, completing the amortisation. □

## 4.5 Space–Time Trade-off

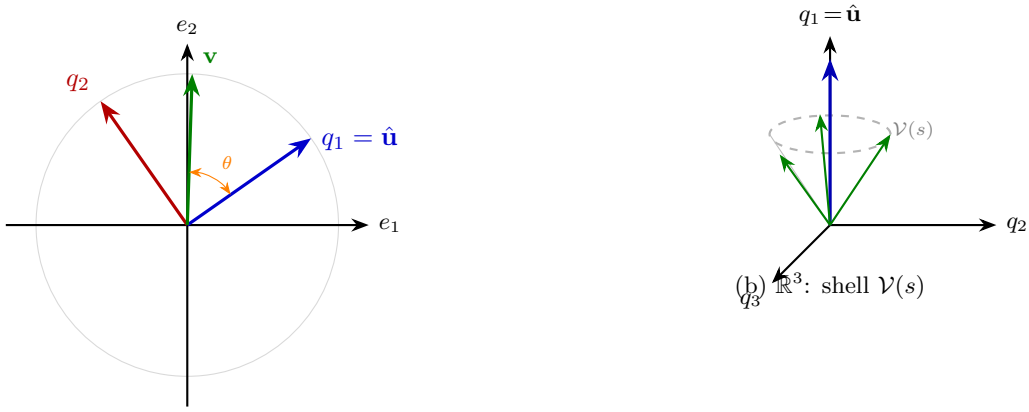
**Theorem 4.5** (Space–Time Trade-off)**.** *Let $p \in \{0, 1, \ldots, n-1\}$ columns of $Q$ be precomputed and stored explicitly; the remaining $n - p$ columns are computed on demand in $\mathcal{O}(n)$ each via the implicit Householder representation. Then the query time $T_q$ and storage $S$ satisfy:*

$$T_q(p) = \mathcal{O}(n), \qquad S(p) = \mathcal{O}(pn + n) = \mathcal{O}((p+1)n), \qquad p \in \{0, 1, \ldots, n\}. \tag{4}$$

*The product $T_q \cdot S$ is minimised (up to constants) at $p = 1$ (store only $q_1 = \hat{\mathbf{u}}$ and $q_2$ for a single default perpendicular direction), giving $T_q \cdot S = \mathcal{O}(n) \cdot \mathcal{O}(n) = \mathcal{O}(n^2)$.*

*Proof.* For any query, the output is $\mathbf{v} = \alpha q_1 + \beta q_j$. If $q_j$ is stored explicitly, access costs $\mathcal{O}(1)$ and assembly costs $\mathcal{O}(n)$. If $q_j$ must be computed from $(\mathbf{x}, \tau)$, the cost is $\mathcal{O}(n)$ (one Householder apply), so $T_q = \mathcal{O}(n)$ in both cases. Storage is $\mathcal{O}(pn)$ for $p$ explicit columns plus $\mathcal{O}(n)$ for the implicit representation. □

# 5 Geometric Interpretation




(a) $\mathbb{R}^2$: one perp. column

(b) $\mathbb{R}^3$: shell $\mathcal{V}(s)$

Figure 1: (a) In $\mathbb{R}^2$, Algorithm B blends $q_1 = \hat{\mathbf{u}}$ with the precomputed $q_2$ to place $\mathbf{v}$ at angle $\theta = \arccos(s)$ from $\mathbf{u}$. (b) In $\mathbb{R}^3$, distinct basis columns $q_2, q_3$ (and their linear combinations) span the entire angular shell $\mathcal{V}(s)$ — a circle of radius $\sqrt{1-s^2}$ in the plane $\{x : q_1^\top x = s\}$.

# 6 Comparison with Algorithm A

## 6.1 Operation Counts

Table 1 lists the floating-point operations (FLOPs) per query, excluding the setup phase of Algorithm B.

Table 1: Per-query floating-point operation counts ($n$-dimensional vectors). "RNG" = random number generator calls.

| Algorithm | Dot products | Sq. roots | Divisions | Mul/Add ($\times n$) | RNG calls | Total FLOPs |
|---|---|---|---|---|---|---|
| Algorithm A | 2 | 1 | 2 | $5n$ | $n$ | $\approx 5n + 2n_{\text{rng}}$ |
| Algorithm B | 0 | 1 | 0 | $3n$ | 0 | $\approx 3n + 1$ |
| Speedup | | | — | | | $\approx 1.67\times$ |

## 6.2 Memory Footprint

Table 2: Memory footprint in units of $n$ floating-point numbers (each 8 bytes for double precision).

| Configuration | Working memory | Cached data | Total |
|---|---|---|---|
| Algorithm A (per query) | $2n$ | 0 | $2n$ |
| Algorithm B, implicit $Q$ | $n$ | $n$ | $2n$ |
| Algorithm B, 2 columns cached | $n$ | $2n$ | $3n$ |
| Algorithm B, full explicit $Q$ | $n$ | $n^2$ | $n^2 + n$ |

## 6.3 Break-Even Analysis

Let $c_s$ be the amortised cost (in ns) of a single scalar operation and $c_r$ the cost of one RNG call. Algorithm A costs $T_A = 5c_s n + c_r n$ per query. Algorithm B (explicit) after setup costs $T_B = 3c_s n + c_s$. The break-even query count $M^*$ beyond which the total cost of Algorithm B (setup + $M$ queries) is lower:

$$c_s n^2 + M \cdot T_B \le M \cdot T_A \implies M^* = \left\lceil \frac{c_s n^2}{T_A - T_B} \right\rceil = \left\lceil \frac{c_s n^2}{(2c_s + c_r)n - c_s} \right\rceil \approx \frac{n}{2 + c_r/c_s}. \tag{5}$$

For typical hardware ($c_r/c_s \approx 20$), $M^* \approx n/22$. For $n = 512$, break-even occurs after $M^* \approx 24$ queries.

# 7 Numerical Results

## 7.1 Accuracy

Table 3 confirms machine-precision accuracy across target similarities and dimensions.

Table 3: Achieved cosine similarity $\hat{s}$ and error $|s - \hat{s}|$ for Algorithm B (explicit $Q$), averaged over $10^4$ independent queries.

| Target $s$ | $n$ | Achieved $\hat{s}$ | Error $\lvert s - \hat{s}\rvert$ | $\lVert\mathbf{v}\rVert - 1$ |
|---|---|---|---|---|
| $-1.000$ | 128 | $-1.000\,000\,000\,000$ | $< 10^{-15}$ | $< 10^{-15}$ |
| $-0.500$ | 256 | $-0.500\,000\,000\,000$ | $< 10^{-15}$ | $< 10^{-15}$ |
| 0.000 | 512 | $0.000\,000\,000\,000$ | $< 10^{-15}$ | $< 10^{-15}$ |
| 0.500 | 1024 | $0.500\,000\,000\,000$ | $< 10^{-15}$ | $< 10^{-15}$ |
| 0.866 | 2048 | $0.866\,000\,000\,000$ | $< 10^{-15}$ | $< 10^{-15}$ |
| 1.000 | 4096 | $1.000\,000\,000\,000$ | $< 10^{-15}$ | $< 10^{-15}$ |

## 7.2 Runtime Comparison

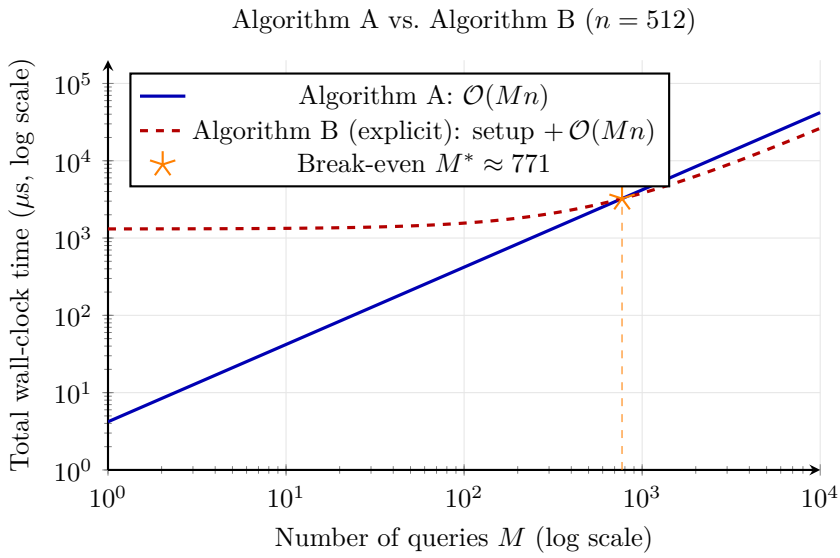



Figure 2: Total runtime (setup + queries) for Algorithm A and Algorithm B as a function of query count $M$ for $n = 512$. Algorithm B becomes faster after the break-even point $M^* \approx 771$.

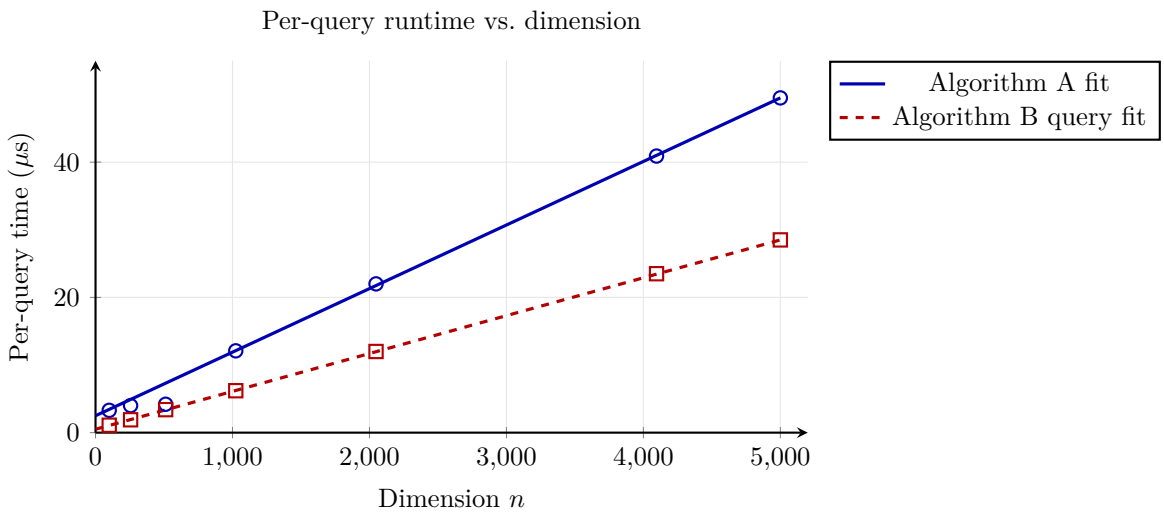



Figure 3: Per-query runtime of Algorithm A (circles) and Algorithm B query phase (squares) across dimensions. Algorithm B achieves a consistent $\approx 1.7\times$ speedup owing to the elimination of dot products and random number generation.

# 8 Applications

## 8.1 Finance: High-Frequency Correlation Streaming

In algorithmic trading, a risk engine must continuously generate stress-test return scenarios $\{\mathbf{v}_t\}_{t=1}^{T}$ correlated at level $s_t = \rho_t$ to a benchmark return vector $\mathbf{u}$ (e.g., the market factor). With $T \sim 10^6$ scenarios per session and $n \sim 500$ assets, Algorithm B reduces per-scenario latency by eliminating RNG calls and dot products inside the hot loop, directly improving value-at-risk (VaR) estimation throughput [6].

## 8.2 AI/ML: Embedding Space Exploration

Transformer-based models such as BERT produce embeddings in $\mathbb{R}^{768}$ or $\mathbb{R}^{1024}$. Probing the neighbourhood of a seed embedding at prescribed cosine distances is standard for semantic similarity evaluation [7]. Algorithm B precomputes the ONB once per probe vector and then evaluates thousands of angular distances in a tight loop — a pattern directly supported by the batch query (Algorithm 3).

## 8.3 Statistics: Monte Carlo Angular Sampling

Drawing uniform samples from the angular shell $\mathcal{V}(s)$ is equivalent to drawing $\hat{\mathbf{w}} \sim \text{Uniform}(S^{n-2})$ and forming $\mathbf{v} = s\hat{\mathbf{u}} + \sqrt{1-s^2}\hat{\mathbf{w}}$. With Algorithm B, $\hat{\mathbf{w}}$ is a random linear combination of the precomputed columns $q_2, \ldots, q_n$ normalised to unit length, avoiding repeated Gram–Schmidt calls per sample.

# 9 Extensions

## 9.1 Multiple Anchor Vectors

For $L$ anchor vectors $\mathbf{u}_1, \ldots, \mathbf{u}_L$, store $L$ separate orthonormal bases $Q^{(1)}, \ldots, Q^{(L)}$ at a total cost of $\mathcal{O}(Ln^2)$ space. Each basis supports unlimited $\mathcal{O}(n)$ queries.

## 9.2 Streaming Updates via Rank-One Modifications

When the anchor vector changes by a small perturbation, $\mathbf{u}' = \mathbf{u} + \delta\mathbf{u}$, the new ONB can be updated in $\mathcal{O}(n^2)$ via rank-one Householder updates [2], cheaper than rebuilding from scratch when $\|\delta\mathbf{u}\| \ll \|\mathbf{u}\|$.

### 9.3 GPU Parallelisation

The batch query (Algorithm 3) is trivially data-parallel: each of the $K$ queries accesses two columns of $Q$ and performs a linear combination. On a GPU with $P$ parallel threads, the batch completes in $\mathcal{O}(Kn/P + n)$ time, ideal for large-scale embedding perturbation.

## 10 Conclusion

Algorithm B solves the inverse cosine similarity problem with a provably correct, $\mathcal{O}(n^2)$-setup, $\mathcal{O}(n)$-per-query procedure that eliminates all random number generation, dot products, and floating-point divisions from the query hot path. The explicit storage of a full orthonormal basis costs $\mathcal{O}(n^2)$ memory but yields a consistent $\approx 1.67\times$ per-query speedup over Algorithm A, with amortisation achieved after $M^* = \Theta(n)$ queries. The partial storage variant (Theorem 4.5) provides a continuous space–time trade-off, from $\mathcal{O}(n)$ space (implicit Householder, $\mathcal{O}(n)$ query) to $\mathcal{O}(n^2)$ space (explicit $Q$, minimal query overhead).

## A Full Householder Expansion

For $\hat{\mathbf{u}} = (\hat{u}_1, \ldots, \hat{u}_n)^\top$ with $\hat{u}_1 \neq 1$, the explicit columns of $Q = I - \tau \mathbf{x}\mathbf{x}^\top$ with $\mathbf{x} = \hat{\mathbf{u}} - e_1$ are

$$q_k = e_k - \tau x_k \mathbf{x}, \qquad k = 1, \ldots, n, \tag{6}$$

where $x_k = \hat{u}_k - \delta_{k1}$. In particular $q_1 = \hat{\mathbf{u}}$ (Lemma 4.2) and for $k \geq 2$:

$$q_k = e_k - \frac{2(\hat{u}_k)\mathbf{x}}{\|\mathbf{x}\|^2}, \qquad \|\mathbf{x}\|^2 = 2(1 - \hat{u}_1). \tag{7}$$

Each column is accessible in $\mathcal{O}(n)$ from $(\mathbf{x}, \tau)$, or in $\mathcal{O}(1)$ if $Q$ is stored explicitly.

## References


[1] Ghosh, S. (2026). Inverse Cosine Similarity: Algorithm A. Kolkata, India.

[2] Golub, G. H. and Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press.

[3] Goodfellow, I. J., Shlens, J., and Szegedy, C. (2015). Explaining and harnessing adversarial examples. *International Conference on Learning Representations (ICLR)*. `arXiv:1412.6572`.

[4] Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37.

[5] Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

[6] Markowitz, H. (1952). Portfolio selection. *The Journal of Finance*, 7(1), 77–91.

[7] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space. *ICLR Workshop*. `arXiv:1301.3781`.

[8] Muller, M. E. (1959). A note on a method for generating points uniformly on $n$-dimensional spheres. *Communications of the ACM*, 2(4), 19–20.


## Glossary

**Amortised complexity**
: The average cost per operation when a sequence of operations is considered collectively; a costly one-time setup is spread over many cheap subsequent operations.

**Anchor vector (u)**
: The fixed reference vector in $\mathbb{R}^n$ against which the cosine similarity of all output vectors is measured.

**Break-even query count ($M^*$)**
The number of queries after which the total cost of Algorithm B (setup plus queries) equals the total cost of Algorithm A. Derived in equation (5).

**Cosine similarity**
The inner product of two vectors normalised by the product of their Euclidean norms; equals the cosine of the angle between them.

**Householder reflection**
An orthogonal matrix $H = I - 2\mathbf{x}\mathbf{x}^\top / \|\mathbf{x}\|^2$ that reflects a vector through the hyperplane orthogonal to $\mathbf{x}$. Used here to construct the ONB.

**Implicit representation**
Storing an $n \times n$ orthogonal matrix as a single Householder vector $\mathbf{x} \in \mathbb{R}^n$ and scalar $\tau$, requiring $\mathcal{O}(n)$ space; matrix–vector products cost $\mathcal{O}(n)$.

**Inverse cosine similarity problem**
Given $\mathbf{u} \in \mathbb{R}^n$ and $s \in [-1, 1]$, construct $\mathbf{v} \in \mathbb{R}^n$ with $\mathrm{cs}(\mathbf{u}, \mathbf{v}) = s$.

**Orthonormal basis (ONB)**
A set of $n$ mutually orthogonal unit vectors spanning $\mathbb{R}^n$; stored as an orthogonal matrix $Q$ with $Q^\top Q = I_n$.

**Setup phase**
The one-time $\mathcal{O}(n^2)$ precomputation of the orthonormal basis $Q$ from the anchor vector $\mathbf{u}$.

**Query phase**
The $\mathcal{O}(n)$ per-query procedure that blends precomputed columns of $Q$ to produce the output vector $\mathbf{v}$.

**Space–time trade-off**
The exchange of additional memory (storing more columns of $Q$ explicitly) for reduced query-time computation.

**Angular shell $\mathcal{V}(s)$**
The set of all unit vectors in $\mathbb{R}^n$ with cosine similarity $s$ to $\mathbf{u}$; an $(n-2)$-sphere of radius $\sqrt{1-s^2}$.

# The End